

LAPORAN PRAKTIKUM 12

Nama : Akmallutfi Afza Gusty

NIM : 707012500064

Kelas : D4 SIKC 49-04

Mata kuliah : Pemrograman Web

Tujuan Praktikum

Setelah menyelesaikan praktikum ini, mahasiswa diharapkan mampu:

- Memahami arsitektur integrasi antara Single Page Application (SPA) berbasis React dengan REST API Backend.
- Mengimplementasikan konsep Routing dinamis menggunakan library 'react-router-dom' v6.
- Menggunakan library 'axios' untuk melakukan request asinkron (HTTP Client) ke REST API backend lokal.
- Menerapkan sistem pengamanan rute internal (Route Guard) melalui pembuatan komponen 'ProtectedRoute'.
- Mengelola state autentikasi secara global menggunakan konsep state sharing React Context ('AuthContext').
- Mengintegrasikan gaya antarmuka responsif dan modern dengan utility classes Tailwind CSS v3.
- Menggunakan sistem pembagian token otorisasi JWT (JSON Web Token) yang disisipkan melalui HTTP request headers.
- Mengekspor aplikasi React produksi via 'npm run build' dan men-deploy-nya ke web server Apache lokal (htdocs).

Alat dan Bahan

Untuk melaksanakan praktikum ini, diperlukan:

- XAMPP Web Server (Layanan Apache dan MySQL aktif secara lokal)
- PHP versi 7.x atau PHP versi 8.x
- REST API Backend berbasis CodeIgniter 3 (Program dari Modul Praktikum 9)
- Node.js runtime environment (untuk npm dan eksekusi Vite)
- Library pendukung Frontend: 'axios' dan 'react-router-dom'
- Framework CSS: Tailwind CSS v3 beserta PostCSS dan Autoprefixer
- Visual Studio Code (sebagai Code Editor)
- Browser modern (Google Chrome / Mozilla Firefox)

Langkah-Langkah Praktikum

Berikut adalah implementasi kode lengkap dari penyelesaian aplikasi React Blog Post. Seluruh kode sumber di bawah ini telah disesuaikan secara dinamis dari proyek praktikum dan dilengkapi dengan komentar penjelas berbahasa Indonesia.

1. Konfigurasi Server Pengembangan (vite.config.js)

File konfigurasi Vite yang telah disesuaikan. Berisi pengaturan base path relatif ('./') untuk mempermudah pemindahan folder dist ke XAMPP htdocs, serta penyetelan Server Proxy untuk membelokkan request '/ci3_project' secara aman ke server Apache lokal (http://localhost) demi menghindari kendala keamanan CORS (Cross-Origin Resource Sharing).

```
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

// https://vite.dev/config/
export default defineConfig({
  // Menentukan base path build agar menggunakan path relatif (memudahkan pemindahan
  // folder dist ke htdocs)
  base: './',
  plugins: [react()],
  // Konfigurasi server pengembangan (development server) Vite
  server: {
    // Proxy digunakan untuk mengarahkan request API dari frontend ke server backend
    // lokal (XAMPP/Apache)
    // Hal ini berguna untuk menghindari kendala keamanan CORS (Cross-Origin
    // Resource Sharing) saat pengembangan
    proxy: {
      '/ci3_project': {
        target: 'http://localhost', // Domain / host server backend lokal Anda
        changeOrigin: true, // Mengubah origin header request agar sesuai dengan
        target
      },
    },
  },
})
```

2. Gaya Global & Integrasi Tailwind (src/index.css)

Gaya global aplikasi. File ini mengintegrasikan direktif dasar Tailwind CSS (base, components, utilities), menetapkan jenis font 'Inter' kustom ke elemen body, serta menyisipkan class loading spinner pendukung untuk transisi loading halaman terproteksi.

```
/* Mengimpor modul dasar (base styles) dari Tailwind CSS */
@tailwind base;

/* Mengimpor komponen pra-desain (component classes) dari Tailwind CSS */
```

```

@tailwind components;

/* Mengimpor utilitas pembantu (utility classes) dari Tailwind CSS */
@tailwind utilities;

:root {
  /* Mengatur skema warna default root agar mendukung tampilan gelap/terang secara
  global */
  color-scheme: dark;
}

body {
  /* Menerapkan font kustom 'Inter' dan fallback sistem font modern untuk tampilan
  premium */
  font-family: 'Inter', -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-
  serif;
}

/*
  Mendefinisikan gaya CSS kustom untuk kontainer pemuatan halaman (loading
  container)
  yang digunakan di dalam komponen ProtectedRoute.
*/
.loading-container {
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: center;
  min-height: 100vh;
  background-color: #f3f4f6; /* Sama dengan bg-gray-100 */
}

/*
  Mendefinisikan spinner animasi putar (loading spinner) menggunakan CSS Keyframes
*/
.loading-spinner {
  width: 3rem; /* Ukuran h-12 */
  height: 3rem; /* Ukuran w-12 */
  border-radius: 9999px; /* rounded-full */
  border-bottom-width: 2px; /* border-b-2 */
  border-color: #3b82f6; /* border-blue-600 */
  animation: spin 1s linear infinite; /* animate-spin */
}

/* Animasi putar spinner */
@keyframes spin {
  from {
    transform: rotate(0deg);
  }
  to {
    transform: rotate(360deg);
  }
}

```

3. Layanan Panggilan API (src/services/api.js)

Modul layanan API menggunakan library Axios. Di sini diatur instance Axios dasar dengan baseURL ke CodeIgniter API. Mengimplementasikan Request Interceptor untuk menyimpan JWT Bearer Token ke header otorisasi, serta Response Interceptor untuk mendeteksi error status 401 (Unauthorized) guna mengarahkan kembali pengguna yang tidak sah secara otomatis ke halaman login.

```
import axios from "axios";

// Menentukan URL dasar untuk REST API CodeIgniter 3.
// Ini akan diarahkan melalui proxy Vite ke http://localhost/ci3_project/api
const API_URL = "/ci3_project/api";

// Membuat instance Axios dengan konfigurasi dasar
const api = axios.create({
  baseURL: API_URL,
  headers: {
    "Content-Type": "application/json",
    Accept: "application/json",
  },
});

// Interceptor Permintaan (Request Interceptor)
// Mengambil token JWT dari localStorage jika ada, lalu menyimpannya ke header
// Authorization
api.interceptors.request.use(
  (config) => {
    const token = localStorage.getItem("token");
    if (token) {
      config.headers.Authorization = `Bearer ${token}`;
    }
    return config;
  },
  (error) => {
    return Promise.reject(error);
  }
);

// Interceptor Respons (Response Interceptor)
// Memeriksa jika respons menghasilkan error 401 (Unauthorized/Token kedaluwarsa
// atau tidak valid).
// Jika iya, hapus token/user dari localStorage dan alihkan pengguna kembali ke
// halaman login.
api.interceptors.response.use(
  (response) => response,
  (error) => {
    if (error.response && error.response.status === 401) {
      const requestUrl = error.config?.url || "";
      // Kecualikan endpoint login dan register agar tidak terjadi perulangan
      // pengalihan
    }
  }
);
```

```

        if (!requestUrl.includes("/login") && !requestUrl.includes("/register")) {
            localStorage.removeItem("token");
            localStorage.removeItem("user");
            window.location.href = "/login";
        }
    }
    return Promise.reject(error);
}
);

// Kumpulan endpoint API untuk Autentikasi Pengguna
export const authAPI = {
    // Melakukan login dengan email & password
    login: (email, password) => api.post("/auth/login", { email, password }),
    // Melakukan pendaftaran user baru
    register: (name, email, password, password_confirmation) =>
        api.post("/auth/register", { name, email, password, password_confirmation }),
    // Melakukan logout dari sistem
    logout: () => api.post("/auth/logout"),
    // Mengambil informasi profil user saat ini
    me: () => api.get("/auth/me"),
};

// Kumpulan endpoint API untuk pengelolaan artikel/Postingan
export const postsAPI = {
    // Mengambil semua artikel dengan pagination default
    getAll: (page = 1, perPage = 10) =>
        api.get(`/post?page=${page}&per_page=${perPage}`),
    // Mengambil artikel spesifik berdasarkan ID
    getById: (id) => api.get(`/post/${id}`),
};

export default api;

```

4. Pengelolaan State Global Autentikasi (src/context/AuthContext.jsx)

Modul penyedia Context React (AuthContext) yang berfungsi sebagai state management global untuk proses masuk log (login), keluar log (logout), dan pengecekan otomatis sesi token login yang disimpan di localStorage saat pertama kali aplikasi dimuat.

```

import { createContext, useContext, useState, useEffect } from 'react';
import { authAPI } from '../services/api';

// Membuat React Context untuk menyimpan state autentikasi secara global
const AuthContext = createContext(null);

// Komponen Provider yang membungkus aplikasi dan menyediakan state autentikasi ke semua komponen anak
export const AuthProvider = ({ children }) => {
    const [user, setUser] = useState(null); // Menyimpan data profil user yang sedang login

```

```

    const [token, setToken] = useState(localStorage.getItem('token')); // Menyimpan
    token JWT
    const [loading, setLoading] = useState(true); // Status pemuatan untuk mengecek
    session login saat awal aplikasi dimuat

    // Efek samping (Effect) untuk memeriksa apakah ada sesi login aktif di
    localStorage saat pertama kali aplikasi dijalankan
    useEffect(() => {
        const checkAuth = async () => {
            const savedToken = localStorage.getItem('token');
            const savedUser = localStorage.getItem('user');

            // Jika token dan profil user tersimpan di localStorage, sinkronkan ke state
            React
            if (savedToken && savedUser) {
                setToken(savedToken);
                setUser(JSON.parse(savedUser));
            }
            setLoading(false); // Selesai melakukan pengecekan sesi login
        };
        checkAuth();
    }, []);

    // Fungsi untuk menangani proses masuk log (login)
    const login = async (email, password) => {
        try {
            // Memanggil API login
            const response = await authAPI.login(email, password);
            const { access_token, user: userData } = response.data;

            // Menyimpan token dan data user ke localStorage agar sesi tetap bertahan saat
            halaman direfresh
            localStorage.setItem('token', access_token);
            localStorage.setItem('user', JSON.stringify(userData));

            // Memperbarui state autentikasi React
            setToken(access_token);
            setUser(userData);

            return { success: true };
        } catch (error) {
            // Menangkap pesan error dari backend atau memberikan pesan default jika gagal
            const message = error.response?.data?.message || 'Login gagal';
            return { success: false, message };
        }
    };

    // Fungsi untuk menangani proses keluar log (logout)
    const logout = async () => {
        try {
            // Memanggil API logout di backend untuk menonaktifkan token
            await authAPI.logout();
        } catch (error) {
            console.error('Logout error:', error);
        } finally {

```

```

        // Selalu bersihkan localStorage dan state React meskipun API logout gagal
        localStorage.removeItem('token');
        localStorage.removeItem('user');
        setToken(null);
        setUser(null);
    }
};

// Menyusun value yang akan disebar ke seluruh aplikasi
const value = {
    user,
    token,
    loading,
    isAuthenticated: !!token, // Status boolean apakah user sudah terautentikasi (!!
mengonversi string token menjadi boolean)
    login,
    logout,
};

return (
    <AuthContext.Provider value={value}>
        {children}
    </AuthContext.Provider>
);
};

// Custom hook kustom 'useAuth' untuk memudahkan pemanggilan AuthContext di komponen
React lainnya
export const useAuth = () => {
    const context = useContext(AuthContext);
    if (!context) {
        throw new Error('useAuth harus digunakan di dalam AuthProvider');
    }
    return context;
};

```

5. Pengaman Akses Rute (src/components/ProtectedRoute.jsx)

Komponen Route Guard (penjaga rute). Komponen ini membungkus halaman internal (PostList, PostDetail) dan memeriksa status autentikasi pengguna secara real-time. Jika pengguna belum terautentikasi (belum login), sistem akan langsung mengalihkan rute secara paksa ke '/login'.

```

import { Navigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

// Komponen ProtectedRoute bertindak sebagai penjaga rute (Route Guard).
// Hanya mengizinkan akses ke komponen anak (children) jika pengguna sudah login.
const ProtectedRoute = ({ children }) => {
    // Mengambil status autentikasi dan loading dari AuthContext
    const { isAuthenticated, loading } = useAuth();

```

```

    // Jika masih dalam proses memuat (misalnya mengecek token di local storage saat
    awal reload halaman)
    if (loading) {
      return (
        <div className="loading-container">
          {/* Spinner pemuatan yang bergaya premium, didefinisikan di index.css */}
          <div className="loading-spinner"></div>
          <p className="mt-4 text-gray-500 font-medium animate-pulse">Loading...</p>
        </div>
      );
    }

    // Jika pengguna tidak terautentikasi (belum login), alihkan rute ke halaman login
    if (!isAuthenticated) {
      return <Navigate to="/login" replace />;
    }

    // Jika sudah terautentikasi, tampilkan komponen anak yang dilindungi
    return children;
  };

export default ProtectedRoute;

```

6. Komponen Avatar Default (src/components/DefaultAvatar.jsx)

Komponen visual sederhana yang me-render ikon siluet pengguna berbasis SVG di dalam lingkaran abu-abu. Komponen ini menerima prop 'className' secara fleksibel agar ukurannya dapat menyesuaikan dengan tata letak navbar maupun kartu postingan.

```

import React from 'react';

// Komponen DefaultAvatar kustom.
// Menampilkan placeholder gambar profil (avatar) berupa lingkaran abu-abu berisi
ikon siluet pengguna (SVG).
// Menerima prop className agar ukurannya atau kelas Tailwind lainnya dapat diatur
secara dinamis.
const DefaultAvatar = ({ className = "w-7 h-7" }) => (
  <div className={` ${className} rounded-full bg-gray-300 flex items-center justify-
center flex-shrink-0`} >
    {/* Ikon siluet pengguna berupa SVG */}
    <svg className="w-4 h-4 text-gray-500" fill="currentColor" viewBox="0 0 20 20"
xmlns="http://www.w3.org/2000/svg">
      <path fillRule="evenodd" d="M10 9a3 3 0 10-6 3 3 0 00 6 3zm-7 9a7 7 0 114
0H3z" clipRule="evenodd"></path>
    </svg>
  </div>
);

export default DefaultAvatar;

```

7. Komponen Bilah Navigasi Utama (src/components/Navbar.jsx)

Header bar atas yang menampilkan logo Blog, navigasi menu, nama user aktif, tombol Logout terintegrasi Context, serta hamburger menu responsif untuk pengguna perangkat mobile (tablet/smartphone).

```
import { useState } from 'react';
import { Link } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import DefaultAvatar from './DefaultAvatar';

// Komponen Navbar (Bilah Navigasi) Utama.
// Ditampilkan di bagian atas halaman internal. Menyediakan akses rute utama, menu
responsif, nama profil user, dan tombol logout.
const Navbar = () => {
  // Mengambil profil user yang sedang aktif dan fungsi logout dari useAuth
  const { user, logout } = useAuth();

  // State untuk melacak apakah menu navigasi mobile sedang terbuka/tertutup
  const [mobileMenuOpen, setMobileMenuOpen] = useState(false);

  // Menangani aksi keluar dari sistem saat tombol Logout diklik
  const handleLogout = async () => {
    await logout();
  };

  return (
    <nav className="bg-white border-b border-gray-200 sticky top-0 z-50 shadow-sm">
      {/* Container utama dengan batas lebar maksimal agar terlihat premium di layar
      lebar */}
      <div className="max-w-screen-xl mx-auto px-4">
        <div className="flex items-center justify-between h-16">

          {/* Bagian Kiri: Logo / Judul Web */}
          <Link to="/posts" className="flex items-center text-xl font-bold text-
gray-900 flex-shrink-0">
            {/* Ikon Logo SVG */}
            <svg className="w-8 h-8 mr-2 text-blue-600" fill="currentColor"
viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg">
              <path d="M5 3a2 2 0 0-2 2v2a2 2 0 02 2h2a2 2 0 02-2V5a2 2 0 0-2-
2H5z" data-bbox="111 225 862 285"/>
            </svg>
            Blog Posts
          </Link>

          {/* Bagian Tengah: Menu Navigasi Desktop (Disembunyikan di layar kecil
          'hidden md:flex') */}
          <div className="hidden md:flex items-center space-x-8 absolute left-1/2
transform -translate-x-1/2">
            <Link to="/posts" className="text-blue-600 font-medium hover:text-blue-
```

```

500 transition-colors">Home</Link>
    <a href="#" className="text-gray-600 font-medium hover:text-gray-900
transition-colors">About</a>
    <a href="#" className="text-gray-600 font-medium hover:text-gray-900
transition-colors">Contact</a>
  </div>

  {/ * Bagian Kanan: Profil Pengguna & Tombol Logout / Tombol Menu Mobile */}
  <div className="flex items-center space-x-4">

    {/ * Info Profil User (Nama + Avatar). Disembunyikan di layar handphone
'hidden sm:flex' */}
    <div className="hidden sm:flex items-center space-x-3">
      <DefaultAvatar className="w-8 h-8" />
      {/ * Menampilkan nama pengguna dari state global auth atau fallback ke
'User' jika belum dimuat */}
      <span className="text-sm font-medium text-gray-900">{user?.name ||
'User'}</span>
    </div>

    {/ * Tombol Logout */}
    <button
      onClick={handleLogout}
      className="text-sm text-white bg-red-500 hover:bg-red-600 font-medium
rounded-lg px-4 py-2 transition-colors"
    >
      Logout
    </button>

    {/ * Tombol Hamburger Menu (Hanya muncul di layar mobile 'md:hidden') */}
    <button
      onClick={() => setMobileMenuOpen(!mobileMenuOpen)}
      className="md:hidden p-2 text-gray-600 hover:text-gray-900"
    >
      {/ * Ikon garis tiga (Hamburger) SVG */}
      <svg className="w-6 h-6" fill="none" stroke="currentColor" viewBox="0
0 24 24" xmlns="http://www.w3.org/2000/svg">
        <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2"
d="M4 6h16M4 12h16M4 18h16"></path>
      </svg>
    </button>
  </div>
</div>

  {/ * Menu Navigasi Mobile (Muncul hanya ketika mobileMenuOpen bernilai true
dan layar bertipe mobile) */}
  {mobileMenuOpen && (
    <div className="md:hidden pb-4 border-t border-gray-200 mt-2 pt-4">
      <div className="flex flex-col space-y-3">
        <Link to="/posts" className="text-blue-600 font-medium">Home</Link>
        <a href="#" className="text-gray-600 hover:text-gray-900">About</a>
        <a href="#" className="text-gray-600 hover:text-gray-900">Contact</a>
      </div>
    </div>
  )}

```

```

        </div>
    </nav>
  );
};

export default Navbar;

```

8. Komponen Kaki Halaman (src/components/Footer.jsx)

Komponen kaki halaman (Footer) statis yang memuat info hak cipta, deskripsi platform singkat, dan tautan sumber daya pendukung eksternal.

```
import React from 'react';

// Komponen Footer (Kaki Halaman)
// Menampilkan informasi hak cipta, deskripsi platform singkat, dan tautan sumber daya.
const Footer = () => {
  return (
    <footer className="bg-white border-t border-gray-200">
      {/* Container utama dengan lebar maksimum terpusat agar terlihat premium */}
      <div className="max-w-screen-xl mx-auto px-4 py-8">
        <div className="md:flex md:justify-between">

          {/* Bagian Kiri: Deskripsi Singkat */}
          <div className="mb-6 md:mb-0">
            <a href="#" className="flex items-center">
              {/* Ikon Logo SVG */}
              <svg className="w-8 h-8 mr-2 text-blue-600" fill="currentColor"
viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg">
                <path d="M5 3a2 2 0 0-2 2v2a2 2 0 02 2h2a2 2 0 02-2v5a2 2 0 00-2-2H5z"
M5 11a2 2 0 00-2 2v2a2 2 0 02 2h2a2 2 0 02-2v-2a2 2 0 00-2-2H5zM11 5a2 2 0
012-2h2a2 2 0 012 2v2a2 2 0 01-2 2h-2a2 2 0 01-2-2V5zM14 11a1 1 0 011 1v1h1a1 1 0
110 2h-1v1a1 1 0 11-2 0v-1h-1a1 1 0 110-2h1v-1a1 1 0 011-1z"></path>
              </svg>
              <span className="text-xl font-bold text-gray-900">Blog Posts</span>
            </a>
            <p className="mt-2 text-sm text-gray-500 max-w-sm">
              A simple blog platform built with React & Laravel.
            </p>
          </div>

          {/* Bagian Kanan: Tautan Navigasi Footer */}
          <div className="grid grid-cols-2 gap-8 sm:gap-16">

            {/* Kolom Sumber Daya */}
            <div>
              <h3 className="mb-4 text-sm font-semibold text-gray-900
uppercase">Resources</h3>
              <ul className="text-gray-500 space-y-2 text-sm">
                <li><a href="#" className="hover:text-gray-900 transition-
```

```

colors">React</a></li>
      <li><a href="#" className="hover:text-gray-900 transition-
colors">Laravel</a></li>
      <li><a href="#" className="hover:text-gray-900 transition-
colors">Tailwind CSS</a></li>
    </ul>
  </div>

  {/* Kolom Ikuti Kami */}
  <div>
    <h3 className="mb-4 text-sm font-semibold text-gray-900
uppercase">Follow us</h3>
    <ul className="text-gray-500 space-y-2 text-sm">
      <li><a href="#" className="hover:text-gray-900 transition-
colors">Github</a></li>
      <li><a href="#" className="hover:text-gray-900 transition-
colors">Twitter</a></li>
    </ul>
  </div>
</div>
</div>

  {/* Garis Pembatas Horisontal */}
  <hr className="my-6 border-gray-200" />

  {/* Hak Cipta */}
  <p className="text-center text-sm text-gray-500">
    © 2024 Blog Posts. All rights reserved.
  </p>
</div>
</footer>
);
};

export default Footer;

```

9. Komponen Kartu Ringkasan Artikel (src/components/PostCard.jsx)

Komponen kartu penampil artikel di beranda. Memiliki fungsi internal 'truncateText' untuk memotong isi artikel yang panjang agar terlihat rapi dan fungsi 'formatDate' untuk menghitung dan memformat tanggal relatif pembuatan artikel (Today, Yesterday, X days ago).

```

import { Link } from 'react-router-dom';
import DefaultAvatar from './DefaultAvatar';

// Komponen PostCard.
// Menampilkan kartu ringkasan artikel di halaman utama.
const PostCard = ({ post }) => {

  // Fungsi pembantu untuk memotong teks artikel agar tidak terlalu panjang di
  halaman utama.

```

```

// Default pemotongan adalah 150 karakter.
const truncateText = (text, maxLength = 150) => {
  if (!text) return '';
  if (text.length <= maxLength) return text;
  return text.substring(0, maxLength) + '...';
};

// Fungsi pembantu untuk memformat tanggal secara relatif (Hari ini, Kemarin, X
hari yang lalu)
const formatDate = (dateString) => {
  if (!dateString) return '14 days ago';
  const date = new Date(dateString);
  const now = new Date();
  // Menghitung selisih waktu dalam milidetik dan mengonversinya menjadi hari
  const diffTime = now - date;
  const diffDays = Math.floor(diffTime / (1000 * 60 * 60 * 24));

  if (diffDays === 0) return 'Today';
  if (diffDays === 1) return 'Yesterday';
  if (diffDays < 0) return 'Just now'; // Menangani jika ada selisih zona waktu
kecil
  return `${diffDays} days ago`;
};

return (
  <article className="p-6 bg-white rounded-lg border border-gray-200 shadow-md
hover:shadow-lg transition-shadow">

    {/* Bagian Atas Kartu: Kategori & Tanggal Pembuatan */}
    <div className="flex justify-between items-center mb-5 text-gray-500">
      {/* Label Tag Kategori */}
      <span className="bg-blue-100 text-blue-800 text-xs font-medium inline-flex
items-center px-2.5 py-0.5 rounded">
        {/* Ikon Kertas/Artikel SVG */}
        <svg className="mr-1 w-3 h-3" fill="currentColor" viewBox="0 0 20 20"
xmlns="http://www.w3.org/2000/svg">
          <path fillRule="evenodd" d="M2 5a2 2 0 012-2h8a2 2 0 012 2v10a2 2 0 02
2H4a2 2 0 01-2-2V5zm3 1h6v4H5V6zm6 6H5v2h6v-2z" clipRule="evenodd"></path>
          <path d="M15 7h1a2 2 0 012 2v5.5a1.5 1.5 0 01-3 0V7z"></path>
        </svg>
        Article
      </span>

      {/* Tanggal Relatif Pembuatan */}
      <span className="text-sm flex items-center">
        {/* Ikon Kalender SVG */}
        <svg className="w-4 h-4 mr-1" fill="currentColor" viewBox="0 0 20 20"
xmlns="http://www.w3.org/2000/svg">
          <path fillRule="evenodd" d="M6 2a1 1 0 00-1 1v1H4a2 2 0 00-2 2v10a2 2 0
002 2h12a2 2 0 00-2-2V6a2 2 0 00-2-2h-1V3a1 1 0 10-2 0v1H7V3a1 1 0 00-1-1zm0
5a1 1 0 002 2h8a1 1 0 100-2H6z" clipRule="evenodd"></path>
        </svg>
        {formatDate(post.created_at)}
      </span>
    </div>

```

```

    {/* Bagian Tengah Kartu: Judul Artikel (Klik untuk ke Halaman Detail) */}
    <h2 className="mb-2 text-2xl font-bold tracking-tight text-gray-900">
      <Link to={` /posts/${post.id}`} className="hover:text-blue-600 transition-
colors">
        {post.title}
      </Link>
    </h2>

    {/* Cuplikan Teks Artikel yang Telah Dipotong */}
    <p className="mb-5 font-light text-gray-500">
      {truncateText(post.article)}
    </p>

    {/* Bagian Bawah Kartu: Info Penulis & Tautan "Read more" */}
    <div className="flex justify-between items-center">
      {/* Identitas Penulis */}
      <div className="flex items-center space-x-3">
        <DefaultAvatar />
        <span className="font-medium text-gray-900">{post.author}</span>
      </div>

      {/* Tautan Selengkapnya */}
      <Link
        to={` /posts/${post.id}`}
        className="inline-flex items-center font-medium text-blue-600 hover:text-
blue-500 transition-colors"
      >
        Read more
        {/* Ikon Panah Kanan SVG */}
        <svg className="ml-2 w-4 h-4" fill="currentColor" viewBox="0 0 20 20"
xmlns="http://www.w3.org/2000/svg">
          <path fillRule="evenodd" d="M10.293 3.293a1 1 0 011.414 0l6 6a1 1 0 010
1.4141-6 6a1 1 0 01-1.414-1.414L14.586 11H3a1 1 0 110-2h11.586l-4.293-4.293a1 1 0
010-1.414z" clipRule="evenodd"></path>
        </svg>
      </Link>
    </div>
  </article>
);
};

export default PostCard;

```

10. Halaman Autentikasi Pengguna (src/pages/Login.jsx)

Formulir masuk log (Login). Mengontrol input teks email dan password, menayangkan indikator loading submit saat memproses login asinkron, dan menyajikan pesan alert kegagalan jika data autentikasi yang diinput tidak terdaftar di database server.

```
import { useState } from 'react';
```

```

import { useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

// Halaman Login (Masuk Log)
// Menyediakan form login pengguna dengan validasi dan interaksi status loading
serta penanganan error.
const Login = () => {
  const [email, setEmail] = useState(''); // State untuk menyimpan input email
  const [password, setPassword] = useState(''); // State untuk menyimpan input
password
  const [error, setError] = useState(''); // State untuk menyimpan pesan
kesalahan/error login
  const [loading, setLoading] = useState(false); // State untuk melacak status
pemuatan asinkron saat submit form

  // Mengambil fungsi login dari useAuth context
  const { login } = useAuth();
  // Navigasi Router DOM untuk mengalihkan rute secara terprogram
  const navigate = useNavigate();

  // Menangani proses submit form
  const handleSubmit = async (e) => {
    e.preventDefault(); // Mencegah reload halaman bawaan browser saat submit form
    setError(''); // Bersihkan pesan kesalahan sebelumnya
    setLoading(true); // Mulai pemuatan

    try {
      // Memanggil fungsi login asinkron dari AuthContext
      const result = await login(email, password);

      if (result.success) {
        // Jika login berhasil, alihkan pengguna ke halaman daftar postingan
        navigate('/posts');
      } else {
        // Jika login gagal (misal kredensial salah), tampilkan pesan kegagalan dari
        setError(result.message);
      }
    } catch (err) {
      console.error('Login error:', err);
      // Fallback jika terjadi error jaringan atau server mati
      setError(err.message || 'Terjadi kesalahan saat login. Pastikan server
      Laravel/CI3 sudah berjalan.');
```

```

<a href="#" className="flex items-center mb-8 text-2xl font-semibold text-
gray-900">
  {/* Ikon Logo SVG */}
  <svg className="w-10 h-10 mr-3 text-blue-600" fill="currentColor"
viewBox="0 0 20 20" xmlns="http://www.w3.org/2000/svg">
    <path d="M5 3a2 2 0 00-2 2v2a2 2 0 002 2h2a2 2 0 002-2V5a2 2 0 00-2-
2H5zM5 11a2 2 0 00-2 2v2a2 2 0 002 2h2a2 2 0 002-2v-2a2 2 0 00-2-2H5zM11 5a2 2 0
012-2h2a2 2 0 012 2v2a2 2 0 01-2 2h-2a2 2 0 01-2-2V5zM14 11a1 1 0 011 1v1h1a1 1 0
110 2h-1v1a1 1 0 11-2 0v-1h-1a1 1 0 110-2h1v-1a1 1 0 011-1z"></path>
  </svg>
  Blog Posts
</a>

  {/* Box Formulir Login */}
  <div className="w-full max-w-md bg-white rounded-xl shadow-lg border border-
gray-200 p-8">
    <h1 className="text-2xl font-bold text-gray-900 mb-6">Sign in to your
account</h1>

    {/* Alert Penampil Kesalahan (Hanya tampil jika ada pesan error) */}
    {error && (
      <div className="mb-6 p-4 text-sm text-red-700 bg-red-100 rounded-lg
border border-red-200" role="alert">
        {error}
      </div>
    )}

    {/* Form Utama */}
    <form className="space-y-6" onSubmit={handleSubmit}>
      {/* Input Email */}
      <div>
        <label htmlFor="email" className="block mb-2 text-sm font-medium text-
gray-900">Your email</label>
        <input
          type="email"
          name="email"
          id="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          className="w-full px-4 py-3 bg-gray-50 border border-gray-300
rounded-lg text-gray-900 placeholder-gray-400 focus:ring-2 focus:ring-blue-500
focus:border-blue-500 outline-none transition-all"
          placeholder="name@company.com"
          required
        />
      </div>

      {/* Input Password */}
      <div>
        <label htmlFor="password" className="block mb-2 text-sm font-medium
text-gray-900">Password</label>
        <input
          type="password"
          name="password"
          id="password"

```

```

        value={password}
        onChange={(e) => setPassword(e.target.value)}
        placeholder="....."
        className="w-full px-4 py-3 bg-gray-50 border border-gray-300
rounded-lg text-gray-900 placeholder-gray-400 focus:ring-2 focus:ring-blue-500
focus:border-blue-500 outline-none transition-all"
        required
      />
    </div>

    {/* Opsi Ingat Saya & Lupa Password */}
    <div className="flex items-center justify-between">
      <div className="flex items-center">
        <input
          id="remember"
          type="checkbox"
          className="w-4 h-4 border border-gray-300 rounded bg-gray-50
focus:ring-2 focus:ring-blue-500"
        />
        <label htmlFor="remember" className="ml-2 text-sm text-gray-
600">Remember me</label>
      </div>
      <a href="#" className="text-sm font-medium text-blue-600 hover:text-
blue-500">Forgot password?</a>
    </div>

    {/* Tombol Kirim Form */}
    <button
      type="submit"
      disabled={loading}
      className="w-full py-3 px-4 bg-blue-600 hover:bg-blue-700 text-white
font-medium rounded-lg transition-colors focus:ring-4 focus:ring-blue-300
disabled:opacity-50 disabled:cursor-not-allowed"
    >
      {loading ? 'Signing in...' : 'Sign in'}
    </button>

    {/* Navigasi Daftar Baru */}
    <p className="text-sm text-gray-600 text-center">
      Don't have an account yet?{' '}
      <a href="#" className="font-medium text-blue-600 hover:text-blue-
500">Sign up</a>
    </p>
  </form>
</div>
</div>

{/* Footer Hak Cipta Bawah Halaman */}
<footer className="bg-white border-t border-gray-200 py-4">
  <p className="text-center text-sm text-gray-500">
    © 2024 Blog Posts. All rights reserved.
  </p>
</footer>
</div>
);

```

```
};  
  
export default Login;
```

11. Halaman Utama Beranda Blog (src/pages/PostList.jsx)

Halaman yang me-render daftar seluruh postingan blog. Halaman ini melakukan request data asinkron ke REST API backend menggunakan Axios saat dimuat (componentDidMount via useEffect), memproses status loading spinner, menampilkan error alert beserta tombol retry jika server mati, dan memetakan postingan ke dalam komponen PostCard.

```
import { useState, useEffect } from 'react';  
import { postsAPI } from '../services/api';  
import Navbar from '../components/Navbar';  
import Footer from '../components/Footer';  
import PostCard from '../components/PostCard';  
  
// Halaman Daftar Postingan (PostList)  
// Halaman utama setelah login yang memuat seluruh artikel dari REST API backend.  
const PostList = () => {  
  const [posts, setPosts] = useState([]); // State untuk menyimpan array daftar  
  artikel  
  const [loading, setLoading] = useState(true); // State untuk melacak status  
  pemuatan data dari API  
  const [error, setError] = useState(''); // State untuk menyimpan pesan error jika  
  pemanggilan API gagal  
  
  // Memanggil fungsi fetchPosts secara otomatis saat halaman pertama kali dimuat  
  useEffect(() => {  
    fetchPosts();  
  }, []);  
  
  // Fungsi asinkron untuk mengambil data daftar artikel dari backend  
  const fetchPosts = async () => {  
    try {  
      setLoading(true); // Aktifkan indikator loading  
      const response = await postsAPI.getAll(); // Memanggil endpoint  
      postsAPI.getAll()  
      setPosts(response.data); // Simpan data postingan ke state posts  
      setError(''); // Hapus pesan error sebelumnya jika ada  
    } catch (err) {  
      setError('Gagal memuat posts. Silakan coba lagi.');// Set pesan kegagalan  
      untuk ditampilkan ke user  
      console.error('Error fetching posts:', err);  
    } finally {  
      setLoading(false); // Matikan indikator loading  
    }  
  };  
  
  return (  
    <div className="min-h-screen bg-gray-100 flex flex-col">
```

```

    {/* Navbar Atas */}
    <Navbar />

    {/* Konten Utama Aplikasi */}
    <main className="flex-1">
      <div className="py-8 px-4 mx-auto max-w-screen-xl lg:py-16 lg:px-6">

        {/* Header/Judul Beranda Blog */}
        <div className="mx-auto max-w-screen-sm text-center lg:mb-16 mb-8">
          <h2 className="mb-4 text-3xl lg:text-4xl tracking-tight font-extrabold text-gray-900">Our Blog</h2>
          <p className="font-light text-gray-500 sm:text-xl">
            Explore our collection of articles about React, web development, and
more.
          </p>
        </div>

        {/*
        Kondisional Rendering berdasarkan status pemuatan:
        1. Jika Loading: Tampilkan Animasi Spinner Putar
        */}
        {loading ? (
          <div className="flex justify-center items-center py-20">
            <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-
blue-600"></div>
          </div>
        ) :
        /*
        2. Jika Terjadi Error: Tampilkan Pesan Error & Tombol Coba Lagi
        */
        error ? (
          <div className="text-center py-20">
            <p className="text-red-500 mb-4">{error}</p>
            <button
              onClick={fetchPosts}
              className="text-white bg-blue-600 hover:bg-blue-700 font-medium rounded-lg text-sm px-5 py-2.5 transition-colors"
            >
              Coba Lagi
            </button>
          </div>
        ) :
        /*
        3. Jika Daftar Artikel Kosong: Tampilkan Info Kosong
        */
        posts.length === 0 ? (
          <div className="text-center py-20">
            <p className="text-gray-500">Belum ada posts.</p>
          </div>
        ) :
        /*
        4. Jika Data Ada: Render dalam bentuk Grid Responsif berisi komponen
PostCard
        */
        (

```

```

        <div className="grid gap-8 lg:grid-cols-2">
          {posts.map((post) => (
            <PostCard key={post.id} post={post} />
          ))}
        </div>
      )}
    </div>
  </main>

  { /* Footer Bawah */}
  <Footer />
</div>
);
};

export default PostList;

```

12. Halaman Rincian Detail Artikel (src/pages/PostDetail.jsx)

Halaman untuk membaca satu artikel blog secara penuh berdasarkan parameter ID dinamis pada rute URL. Mengimplementasikan pemisahan teks artikel per baris baru ("\n"), menangani secara eksplisit error 404 jika artikel tidak ditemukan, serta otomatis menyembunyikan elemen gambar header via penanganan event 'onError' jika URL gambar rusak.

```

import { useState, useEffect } from 'react';
import { useParams, Link } from 'react-router-dom';
import { postsAPI } from '../services/api';
import Navbar from '../components/Navbar';
import Footer from '../components/Footer';
import DefaultAvatar from '../components/DefaultAvatar';

// Halaman Detail Artikel (PostDetail)
// Menampilkan artikel secara lengkap berdasarkan ID rute spesifik yang diakses.
const PostDetail = () => {
  // Mengambil parameter ID dari URL menggunakan useParams hook
  const { id } = useParams();
  const [post, setPost] = useState(null); // State untuk menyimpan data tunggal artikel
  const [loading, setLoading] = useState(true); // State untuk melacak status pemuatan asinkron
  const [error, setError] = useState(''); // State untuk menampung pesan kesalahan

  // Memanggil fungsi fetchPost secara otomatis saat ID halaman berubah
  useEffect(() => {
    fetchPost();
  }, [id]);

  // Fungsi asinkron untuk mengambil satu artikel berdasarkan ID dari API
  const fetchPost = async () => {
    try {
      setLoading(true);

```

```

    const response = await postsAPI.getById(id);
    setPost(response.data); // Simpan objek artikel tunggal ke state
    setError('');
  } catch (err) {
    // Memeriksa status error. Jika 404, tampilkan pesan 'tidak ditemukan'
    if (err.response?.status === 404) {
      setError('Post tidak ditemukan.');
```

} else {

```
      setError('Gagal memuat post. Silakan coba lagi.');
```

}

```
    console.error('Error fetching post:', err);
  } finally {
    setLoading(false);
  }
};

// Memformat string tanggal mentah menjadi format lokal bahasa Indonesia (id-ID)
// Contoh: "21 Mei 2026"
const formatDate = (dateString) => {
  if (!dateString) return '';
  const date = new Date(dateString);
  return date.toLocaleDateString('id-ID', {
    day: 'numeric',
    month: 'long',
    year: 'numeric'
  });
};

// 1. Kondisional Rendering saat Loading: Tampilkan spinner pemutar
if (loading) {
  return (
    <div className="min-h-screen bg-gray-100 flex items-center justify-center">
      <div className="animate-spin rounded-full h-12 w-12 border-b-2 border-blue-600"></div>
    </div>
  );
}

// 2. Kondisional Rendering saat Error (Termasuk 404): Tampilkan pesan error & tombol kembali
if (error) {
  return (
    <div className="min-h-screen bg-gray-100 flex flex-col">
      <Navbar />
      <div className="flex-1 flex items-center justify-center py-8 px-4 mx-auto max-w-screen-xl lg:py-16 lg:px-6">
        <div className="text-center py-20">
          <p className="text-red-500 mb-4 font-semibold text-lg">{error}</p>
          <Link
            to="/posts"
            className="text-white bg-blue-600 hover:bg-blue-700 font-medium rounded-lg text-sm px-5 py-2.5 transition-colors"
          >
            ← Kembali ke Daftar Posts
          </Link>
        </div>
      </div>
    </div>
  );
}

```

```

        </div>
    </div>
    <Footer />
</div>
);
}

// 3. Kondisional Rendering ketika Data Artikel Ada
return (
    <div className="min-h-screen bg-gray-100 flex flex-col">
        { /* Navbar Atas */ }
        <Navbar />

        { /* Bagian Utama Artikel */ }
        <main className="flex-1">
            <div className="py-8 px-4 mx-auto max-w-screen-lg lg:py-16 lg:px-6">

                { /* Tombol Navigasi Kembali ke Daftar Post */ }
                <div className="mb-6">
                    <Link
                        to="/posts"
                        className="inline-flex items-center text-sm font-medium text-blue-600
hover:text-blue-500 transition-colors"
                    >
                        { /* Ikon Panah Kiri SVG */ }
                        <svg className="mr-2 w-4 h-4" fill="currentColor" viewBox="0 0 20 20"
xmlns="http://www.w3.org/2000/svg">
                            <path fillRule="evenodd" d="M9.707 16.707a1 1 0 01-1.414 0l-6-6a1 1
0 010-1.414l6-6a1 1 0 011.414 0l6 6a1 1 0 010 1.414l-6 6a1 1 0 01-1.414 0l6 6a1 1 0
010 1.414z" clipRule="evenodd"></path>
                        </svg>
                        Back to Posts
                    </Link>
                </div>

                { /* Lembar Artikel */ }
                <article className="bg-white rounded-lg border border-gray-200 shadow-md
overflow-hidden">
                    { /*
                        Jika postingan memiliki URL gambar, tampilkan gambar header.
                        Gunakan event handler 'onError' untuk menyembunyikan elemen gambar
                        secara otomatis jika URL gambar rusak/tidak valid.
                    */ }
                    {post?.image_url && (
                        <img
                            src={post.image_url}
                            alt={post.title}
                            className="w-full h-64 lg:h-96 object-cover"
                            onError={ (e) => {
                                e.target.style.display = 'none'; // Sembunyikan gambar jika gagal
                                memuat
                            } }
                        />
                    ) }
                </article>
            </div>
        </main>
    </div>
)

```

```

        { /* Bagian Isi Artikel */ }
        <div className="p-6 lg:p-10">
            { /* Baris Informasi Metadata Artikel (Kategori, Penulis, Tanggal
Terbit) */ }
            <div className="flex flex-wrap items-center gap-4 mb-6">
                <span className="bg-blue-100 text-blue-800 text-xs font-medium
inline-flex items-center px-2.5 py-0.5 rounded">
                    <svg className="mr-1 w-3 h-3" fill="currentColor" viewBox="0 0 20
20" xmlns="http://www.w3.org/2000/svg">
                        <path fillRule="evenodd" d="M2 5a2 2 0 012-2h8a2 2 0 012 2v10a2
2 0 002 2H4a2 2 0 01-2-2V5zm3 1h6v4H5V6zm6 6H5v2h6v-2z" clipRule="evenodd"></path>
                        <path d="M15 7h1a2 2 0 012 2v5.5a1.5 1.5 0 01-3 0V7z"></path>
                    </svg>
                    Article
                </span>

                { /* Penulis */ }
                <div className="flex items-center space-x-2">
                    <DefaultAvatar />
                    <span className="font-medium text-sm text-gray-
700">{post?.author}</span>
                </div>

                { /* Tanggal Terbit Terformat */ }
                {post?.created_at && (
                    <span className="text-sm text-gray-500 flex items-center">
                        <svg className="w-4 h-4 mr-1" fill="currentColor" viewBox="0 0
20 20" xmlns="http://www.w3.org/2000/svg">
                            <path fillRule="evenodd" d="M6 2a1 1 0 00-1 1v1H4a2 2 0 00-2
2v10a2 2 0 002 2h12a2 2 0 002-2V6a2 2 0 00-2-2h-1V3a1 1 0 10-2 0v1H7V3a1 1 0 00-1-
1zm0 5a1 1 0 000 2h8a1 1 0 100-2H6z" clipRule="evenodd"></path>
                        </svg>
                        {formatDate(post.created_at)}
                    </span>
                )}
            </div>

            { /* Judul Artikel Besar */ }
            <h1 className="mb-6 text-3xl lg:text-4xl font-bold tracking-tight
text-gray-900">
                {post?.title}
            </h1>

            { /*
            Deskripsi Artikel Lengkap
            Memecah teks paragraf berdasarkan karakter baris baru ('\n')
            lalu memetakannya ke tag paragraf HTML (<p>) agar paragraf rapi dan
terpisah secara visual.
            */ }
            <div className="text-gray-600 leading-relaxed text-lg">
                {post?.article?.split('\n').map((paragraph, index) => (
                    <p key={index} className="mb-4">{paragraph}</p>
                ))}
            </div>
        </div>

```

```

        </article>
      </div>
    </main>

    {/* Footer Bawah */}
    <Footer />
  </div>
);
};

export default PostDetail;

```

13. Pengaturan Perutean Aplikasi (src/App.jsx)

Entrypoint utama rute aplikasi berbasis React Router DOM v6. Seluruh rute aplikasi didefinisikan di sini. Rute publik diarahkan ke '/login', sedangkan rute internal dibungkus oleh komponen 'ProtectedRoute' demi keamanan data. Dilengkapi catch-all route untuk menangkap path tidak dikenal.

```

import React from 'react';
import { BrowserRouter as Router, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider } from '../context/AuthContext';
import ProtectedRoute from '../components/ProtectedRoute';
import Login from '../pages/Login';
import PostList from '../pages/PostList';
import PostDetail from '../pages/PostDetail';
import './index.css';

// Komponen Aplikasi Utama
// Mendefinisikan penyedia state autentikasi (AuthProvider) dan struktur navigasi
rute (react-router-dom)
function App() {
  return (
    // Membungkus seluruh aplikasi dengan AuthProvider agar state login dapat
    diakses di rute mana pun
    <AuthProvider>
      <Router>
        <Routes>
          {/*
            Rute Login (Akses Publik)
            Menyajikan halaman form masuk bagi pengguna anonim
          */}
          <Route path="/login" element={<Login />} />

          {/*
            Rute Terproteksi (Akses Terbatas hanya untuk pengguna yang telah Login)
            Komponen target dibungkus di dalam <ProtectedRoute> untuk mencegah akses
            tidak sah.
          */}
          <Route
            path="/posts"

```

```

        element={
          <ProtectedRoute>
            <PostList />
          </ProtectedRoute>
        }
      />
    <Route
      path="/posts/:id"
      element={
        <ProtectedRoute>
          <PostDetail />
        </ProtectedRoute>
      }
    />

    {/*
      Pengalihan Rute Akar (Root Redirect)
      Jika pengguna mengakses '/' (halaman utama tanpa path), alihkan otomatis
      ke '/posts'.
      Jika sudah login, ProtectedRoute di '/posts' akan menampilkannya. Jika
      belum, akan dilempar ke '/login'.
    */}
    <Route path="/" element={<Navigate to="/posts" replace />} />

    {/*
      Rute Pengalihan 404 (Catch-all Redirect)
      Menangkap semua path yang tidak dikenal/terdaftar di rute, lalu
      mengalihkannya secara aman ke '/posts'
    */}
    <Route path="*" element={<Navigate to="/posts" replace />} />
  </Routes>
</Router>
</AuthProvider>
);
}

export default App;

```

OutPut

Blog Posts

Sign in to your account

Your email

name@company.com

Password

☐ Remember me

[Forgot password?](#)

Sign in

Don't have an account yet? [Sign up](#)

© 2024 Blog Posts. All rights reserved.

Blog Posts

[Home](#) [About](#) [Contact](#)

 [Admin InkuSkill](#) [Logout](#)

Our Blog

Explore our collection of articles about React, web development, and more.

 Article

 21 days ago

test

percobaan pertama

 satu

[Read more →](#)

 Article

 21 days ago

foto kucing - kucing

kucing ini agak garang dan ngeri

 Kucing

[Read more →](#)

 Article

 21 days ago

ini saya

ini adalah saya

 saya

[Read more →](#)

Blog Posts

A simple blog platform built with React & Laravel.

RESOURCES

[React](#)
[Laravel](#)
[Tailwind CSS](#)

FOLLOW US

[Github](#)
[Twitter](#)

© 2024 Blog Posts. All rights reserved.

Analisis dan Pembahasan

1. Arsitektur Client-Server Decoupled (Terpisah):
Praktikum Modul 12 ini mengimplementasikan konsep modern pemisahan arsitektur sistem (decoupled architecture) di mana backend (Server) dan frontend (Client) berjalan secara independen. Backend dikembangkan berbasis web framework klasik PHP CodeIgniter 3 (hasil pengerjaan Modul 9) yang bertindak murni sebagai RESTful API Server yang menghasilkan data berformat terstruktur JSON. Sementara itu, frontend dibangun sebagai Single Page Application (SPA) menggunakan framework modern React JS dengan tool bundler Vite. Komunikasi antar kedua entitas terpisah ini terjalin melalui protokol HTTP dengan bantuan library Axios Client. Arsitektur semacam ini sangat efisien karena mengurangi beban rendering di sisi server, menghemat bandwidth, dan memungkinkan scalability yang fleksibel di mana frontend dan backend dapat diubah atau ditingkatkan tanpa saling memengaruhi satu sama lain.
2. Mekanisme Keamanan Stateful Token Otorisasi (JWT):
Proses autentikasi dalam aplikasi ini memanfaatkan JWT (JSON Web Token) yang disinkronkan secara asinkron. Ketika pengguna men-submit form login di React, permintaan dikirimkan ke server backend lokal. Jika data kredensial valid, server mengembalikan response data user beserta string token JWT. Token ini kemudian disimpan di sisi client menggunakan teknologi 'localStorage'. Untuk setiap permintaan data terproteksi berikutnya (seperti memuat artikel blog), token otorisasi tersebut disematkan ke dalam HTTP request header secara otomatis dengan format 'Authorization: Bearer <token>' melalui fitur Axios Request Interceptor. Jika token kedaluwarsa atau tidak valid, Axios Response Interceptor mendeteksi kesalahan error status 401 dari server, lalu seketika membersihkan localStorage dan mengarahkan paksa rute pengguna kembali ke halaman login. Ini menjamin data internal tidak dapat diakses secara ilegal.
3. Sistem Pengamanan Rute (Route Guarding & State Management):
Keamanan navigasi diatur secara rapi di sisi client menggunakan kombinasi React Context (AuthProvider) dan komponen penjaga rute (ProtectedRoute). AuthProvider bertindak sebagai 'single source of truth' untuk state autentikasi global yang dapat diakses di bagian rute manapun melalui custom hook 'useAuth'. Komponen ProtectedRoute berperan memblokir komponen anak (seperti daftar artikel atau detail artikel) jika state isAuthenticated bernilai false (belum login). Selain itu, ProtectedRoute menangani kondisi sinkronisasi awal di mana aplikasi sedang mengecek session aktif di localStorage saat reload halaman. Selama loading pengecekan bernilai true, sebuah loading spinner premium akan di-render ke layar sebelum mengarahkan rute. Ini mencegah flash halaman tidak sah (unauthorized view leaks).
4. Optimalisasi Kompilasi Produksi (Production Build & Proxy):
Selama proses pengembangan, frontend berjalan di port 5173 sedangkan server Apache backend berjalan di port default 80. Untuk menghindari masalah CORS (Cross-Origin Resource

Sharing) akibat perbedaan origin ini, diatur setup Server Proxy pada 'vite.config.js' yang memetakan path '/ci3_project' langsung ke target 'http://localhost'. Saat kompilasi akhir produksi dijalankan via perintah 'npm run build', seluruh aset JSX dan styling Tailwind v3 dikompilasi (bundled) menjadi berkas statis super ringan HTML, CSS kustom, dan JS murni di dalam direktori 'dist'. Base path relatif ('./') yang disetel pada konfigurasi Vite menjamin aset statis dapat di-deploy dengan sangat mudah di web server mana saja (seperti memindahkannya langsung ke folder htdocs XAMPP) dan dapat diakses dengan andal di URL <http://localhost/react-blog/>.

Kesimpulan

Dari pelaksanaan praktikum Modul 12 ini, dapat ditarik beberapa kesimpulan sebagai berikut:

- Pemisahan Frontend (React) dan Backend (CodeIgniter REST API) merupakan arsitektur modern yang meningkatkan efisiensi rendering data di sisi client dan mempercepat kecepatan pemuatan situs.
- Sistem autentikasi stateless berbasis token JWT (JSON Web Token) yang disinkronkan dengan localStorage memberikan opsi keamanan yang andal dan kompatibel dengan arsitektur RESTful API.
- Pengamanan rute di sisi client (Route Guard) menggunakan React Context dan ProtectedRoute sangat penting dalam SPA untuk memblokir akses ke view internal sebelum user melakukan login sah.
- Axios Interceptors sangat membantu untuk menyederhanakan kode HTTP request dengan otomatis menyisipkan Bearer token JWT di header permintaan dan menangani respons global error secara terpusat (seperti error 401 Unauthorized).
- Framework Tailwind CSS terbukti sangat tangguh dalam mempercepat proses perancangan layout premium yang responsif dan interaktif melalui utility classes terpadu.
- Proses bundling produksi menggunakan bundler modern Vite menghasilkan file build statis (HTML/CSS/JS) di folder 'dist' yang sangat optimal dan mudah di-deploy di web server lokal mana pun (seperti XAMPP htdocs).